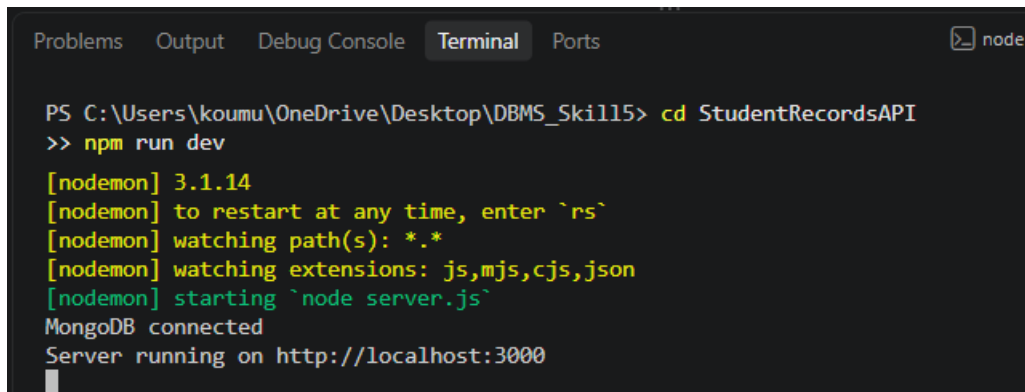


DBMS Practical – 7

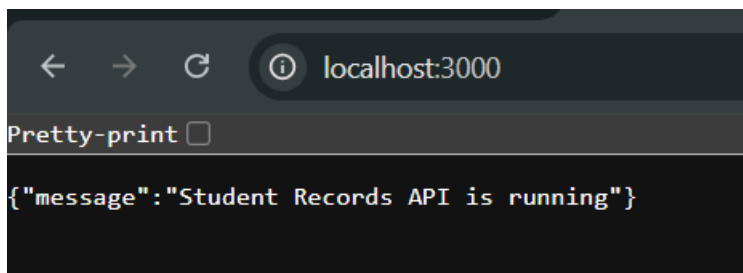
SERVER running:



```
PS C:\Users\koumu\OneDrive\Desktop\DBMS_Skill15> cd StudentRecordsAPI
>> npm run dev

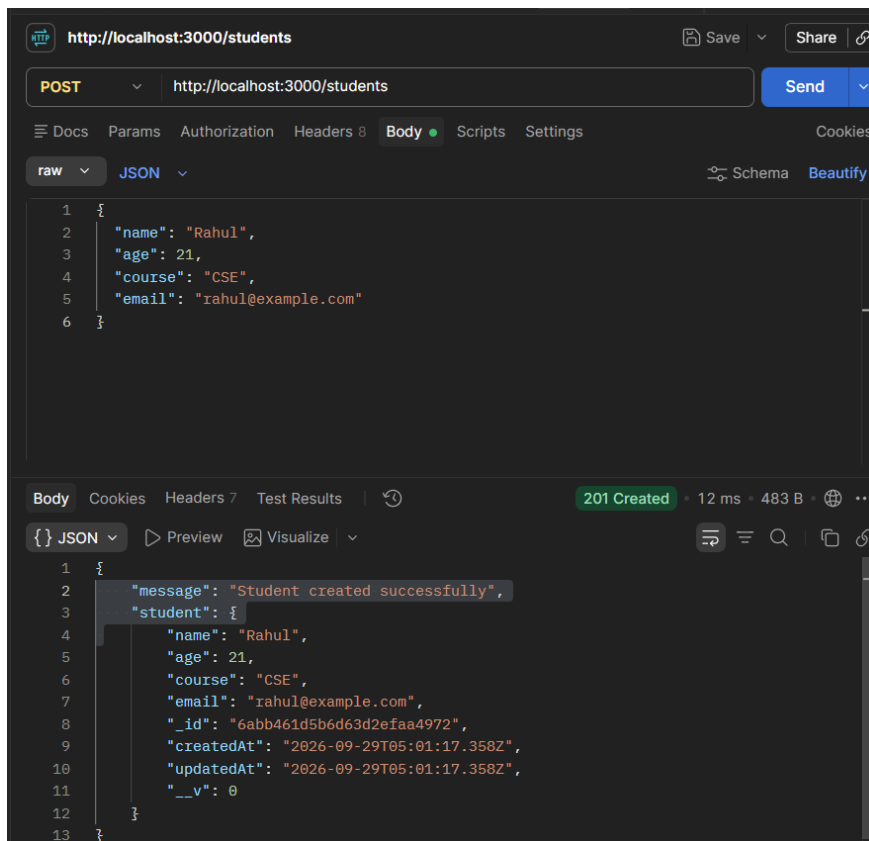
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
MongoDB connected
Server running on http://localhost:3000
```

API Running:



```
{\"message\": \"Student Records API is running\"}
```

POST:



```
http://localhost:3000/students
POST http://localhost:3000/students
{
  "name": "Rahul",
  "age": 21,
  "course": "CSE",
  "email": "rahul@example.com"
}

201 Created - 12 ms - 483 B
{
  "message": "Student created successfully",
  "student": {
    "name": "Rahul",
    "age": 21,
    "course": "CSE",
    "email": "rahul@example.com",
    "_id": "6abb461d5b6d63d2efaa4972",
    "createdAt": "2026-09-29T05:01:17.358Z",
    "updatedAt": "2026-09-29T05:01:17.358Z",
    "__v": 0
  }
}
```

GET:

The screenshot shows a Postman interface for a GET request to `http://localhost:3000/students`. The request body is a JSON object representing a student: `{ "name": "Rahul", "age": 21, "course": "CSE", "email": "rahul@example.com" }`. The response body is a JSON array of two student objects. The first object is partially obscured by a redacted box, but the second object is visible: `{ "_id": "6abb461d5b6d63d2efaa4972", "name": "Rahul", "age": 21, "course": "CSE", "email": "rahul@example.com", "createdAt": "2026-09-29T05:01:17.358Z", "updatedAt": "2026-09-29T05:01:17.358Z", "v": 0 }`.

```
1 {
2   "name": "Rahul",
3   "age": 21,
4   "course": "CSE",
5   "email": "rahul@example.com"
6 }
```

```
1 [
2   {
3     "_id": "6abb42ae5b6d63d2efaa496f",
4     "name": "Anita",
5     "age": 20,
6     "course": "CSE",
7     "email": "anita@example.com",
8     "createdAt": "2026-09-29T04:46:38.600Z",
9     "updatedAt": "2026-09-29T04:46:38.600Z",
10    "__v": 0
11  },
12  {
13    "_id": "6abb461d5b6d63d2efaa4972",
14    "name": "Rahul",
15    "age": 21,
16    "course": "CSE",
17    "email": "rahul@example.com",
18    "createdAt": "2026-09-29T05:01:17.358Z",
19    "updatedAt": "2026-09-29T05:01:17.358Z",
20    "v": 0
21  }
22 ]
```

The screenshot shows a Postman interface for a GET request to `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`. The request body is a JSON object representing a student: `{ "name": "Rahul", "age": 21, "course": "CSE", "email": "rahul@example.com" }`. The response body is a JSON object representing the student with the specified ID, including additional fields like `_id`, `createdAt`, `updatedAt`, and `__v`: `{ "_id": "6abb461d5b6d63d2efaa4972", "name": "Rahul", "age": 21, "course": "CSE", "email": "rahul@example.com", "createdAt": "2026-09-29T05:01:17.358Z", "updatedAt": "2026-09-29T05:01:17.358Z", "v": 0 }`.

```
1 {
2   "name": "Rahul",
3   "age": 21,
4   "course": "CSE",
5   "email": "rahul@example.com"
6 }
```

```
1 {
2   "_id": "6abb461d5b6d63d2efaa4972",
3   "name": "Rahul",
4   "age": 21,
5   "course": "CSE",
6   "email": "rahul@example.com",
7   "createdAt": "2026-09-29T05:01:17.358Z",
8   "updatedAt": "2026-09-29T05:01:17.358Z",
9   "__v": 0
10 }
```

PATCH:

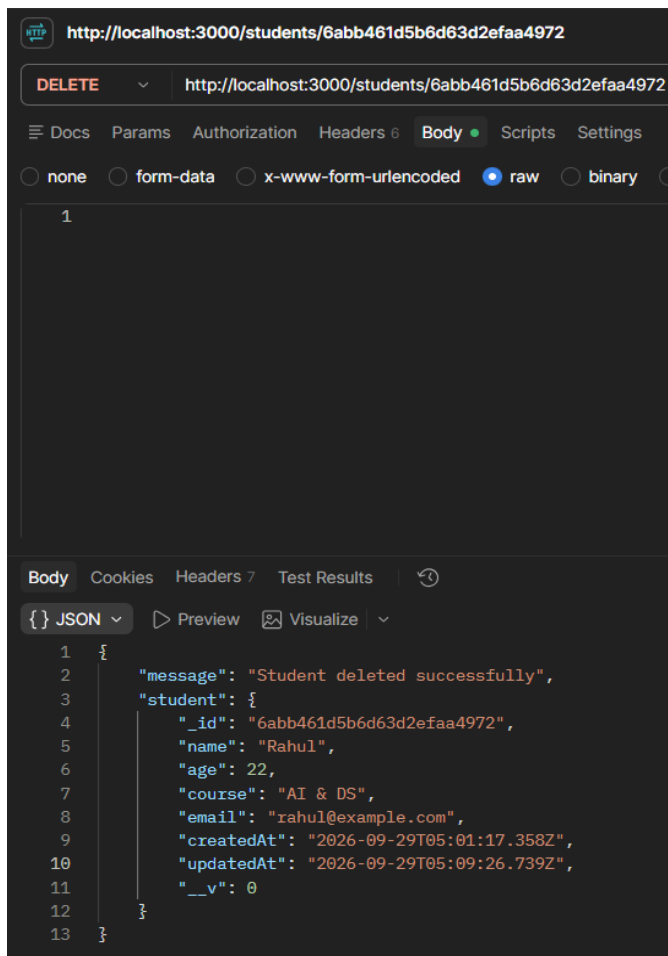
The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`
- Method:** `PATCH`
- Body Type:** `raw` (selected)
- Request Body:**

```
1 {  
2   "age": 22,  
3   "course": "AI & DS"  
4 }
```
- Response Body:** `JSON` (selected)

```
1 {  
2   "message": "Student updated successfully",  
3   "student": {  
4     "_id": "6abb461d5b6d63d2efaa4972",  
5     "name": "Rahul",  
6     "age": 22,  
7     "course": "AI & DS",  
8     "email": "rahul@example.com",  
9     "createdAt": "2026-09-29T05:01:17.358Z",  
10    "updatedAt": "2026-09-29T05:07:57.556Z",  
11    "__v": 0  
12  }  
13 }
```

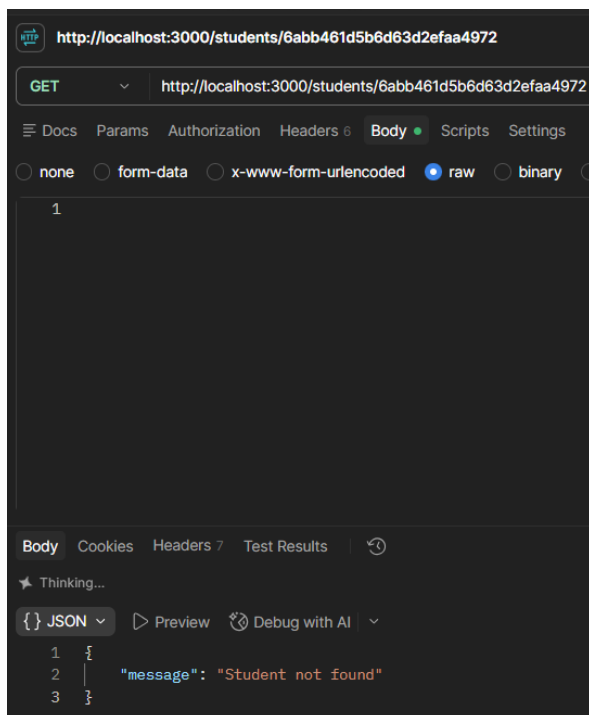
DELETE:



The image shows the Postman interface for a DELETE request. The URL bar at the top displays `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`. Below the URL bar, the HTTP method is set to **DELETE**. The 'Body' tab is selected, and the content type is set to **raw**. The body of the request is empty. The response section at the bottom shows a **200 OK** status with a JSON body:

```
{  "message": "Student deleted successfully",  "student": {    "_id": "6abb461d5b6d63d2efaa4972",    "name": "Rahul",    "age": 22,    "course": "AI & DS",    "email": "rahul@example.com",    "createdAt": "2026-09-29T05:01:17.358Z",    "updatedAt": "2026-09-29T05:09:26.739Z",    "__v": 0  }  }
```

GET (VERIFY):



The image shows the Postman interface for a GET request. The URL bar at the top displays `http://localhost:3000/students/6abb461d5b6d63d2efaa4972`. Below the URL bar, the HTTP method is set to **GET**. The 'Body' tab is selected, and the content type is set to **raw**. The body of the request is empty. The response section at the bottom shows a **200 OK** status with a JSON body:

```
{  "message": "Student not found"}
```

Duplicate Email Validation:

Postman interface showing a POST request to `http://localhost:3000/students`. The request body is a JSON object:

```
{
  "name": "Another Anita",
  "age": 22,
  "course": "ECE",
  "email": "anita@example.com"
}
```

The response is a **400 Bad Request** with a status of 6 ms and 416 B. The response body is a JSON object:

```
{
  "message": "Failed to create student",
  "error": "E11000 duplicate key error collection: student_records.students index: email_1 dup key: { email: \"anita@example.com\" }"
}
```

MongoDB Compass:

MongoDB Compass interface showing the `students` collection in the `student_records` database. The `Documents` tab is active, displaying a single document:

```
{
  "_id": ObjectId('6abb42ae5b6d63d2efaa496f'),
  "name": "Anita",
  "age": 20,
  "course": "CSE",
  "email": "anita@example.com",
  "createdAt": ISODate('2026-09-29T04:46:38.600+00:00'),
  "updatedAt": ISODate('2026-09-29T04:46:38.600+00:00'),
  "__v": 0
}
```